IB113 Úvod do programování a algoritmizace

Cvičení 1

Jaromír Plhák





Cíle předmětu

- Motivovat k programování
- Naučit se řešit problémy a algoritmicky myslet
- Vysvětlit všeobecné principy programování
  - Jak? Psát a číst kód!
- Ilustrovat základní konstrukce v jazyku Python
  - Ne detaily jazyka

Organizace předmětu

- Vše potřebné na stránce
  - https://www.fi.muni.cz/~xpelanek/IB113/
  - https://www.fi.muni.cz/~xpelanek/IB113/sbirka/
- Povinná účast, ale nutno programovat i doma
  - Pokud dojde k nějaké nesrovnalosti, informujte mne prosím mailem
  - Maximálně dvě neomluvené neúčasti
- Vnitrosemestrální test bude 8. (krátký, 50 bodů) a 13. týden (dlouhý, 100 bodů) výuky
  - Není minimální limit, ale …
- Hodnocení
  - 100 bodů z 200 u závěrečné zkoušky
  - 100 bodů z 150 za domácí úkoly (alespoň 5 bodů za každý)
  - 275 bodů celkem

Jak budou probíhat cvičení (1)

- 0. V ISu budou od pátku dostupné prezentace s tím, co se bude v následující hodině procvičovat
  - Opakování toho, co se probíralo na přednášce a co je klíčové pro dané cvičení
  - Prezentaci nebudu detailně procházet
- 1. Kahoot.it
  - Rozcvička, možnost získat bonusové body
- 2. Diskuze - dotazy k učivu shrnutému v prezentaci a zopakování nejdůležitějších věcí
- 3. Samostatné řešení jednoduchého příkladu

Jak budou probíhat cvičení (2)

- 4. Společné (komentované) vyřešení jednoduchého příkladu
- 5. Samostatné řešení příkladů
  - Cca dva příklady dle jejich složitosti
  - Hlavní náplň cvičení
  - Nebojte se pokládat dotazy v průběhu řešení
- [6.] Zadání/Vyhodnocení domácího úkolu
  - Představení zadání/vzorového řešení
  - Možnost diskuze a dotazů ohledně domácího úkolu

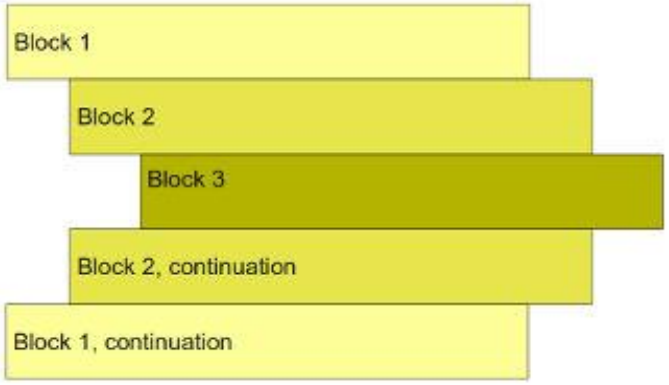
Výhody a nevýhody Pythonu

+ Jednoduchá syntaxe
+ Minimum kódu (téměř pseudokód)
+ Dobrá čitelnost kódu
+ Rozsáhlá standardní knihovna
+ Nezávislost na platformě

− Nemožnost kontroly datových typů při překladu
− Oproti staticky a explicitně typovaným jazykům náchylnost k chybám programátora

Syntaxe Pythonu (1)

- Case sensitive (ne cASe INSensiTIvE)
- Konec příkazu: konec řádku (ne ;)
- Řádkový komentář: #
- Blokový komentář: """
- Bloky kódu: odsazením (ne {} ani begin. . . end)




```
from math import sqrt
n = input("Maximal Number? ")
n = int(n)+1
for a in range(1,n):
    for b in range(a,n):
        c_square = a**2 + b**2
        c = int(sqrt(c_square))
        if ((c_square - c**2) == 0):
            print(a, b, c)
```

Syntaxe Pythonu (2)

- Méně znaků, přirozená čitelnost
  - Není nutné používat všude závorky
- Dobrý zvyk: po dvojtečce nový řádek a 4 mezery
  - Nastavení znaku Tab v IDE

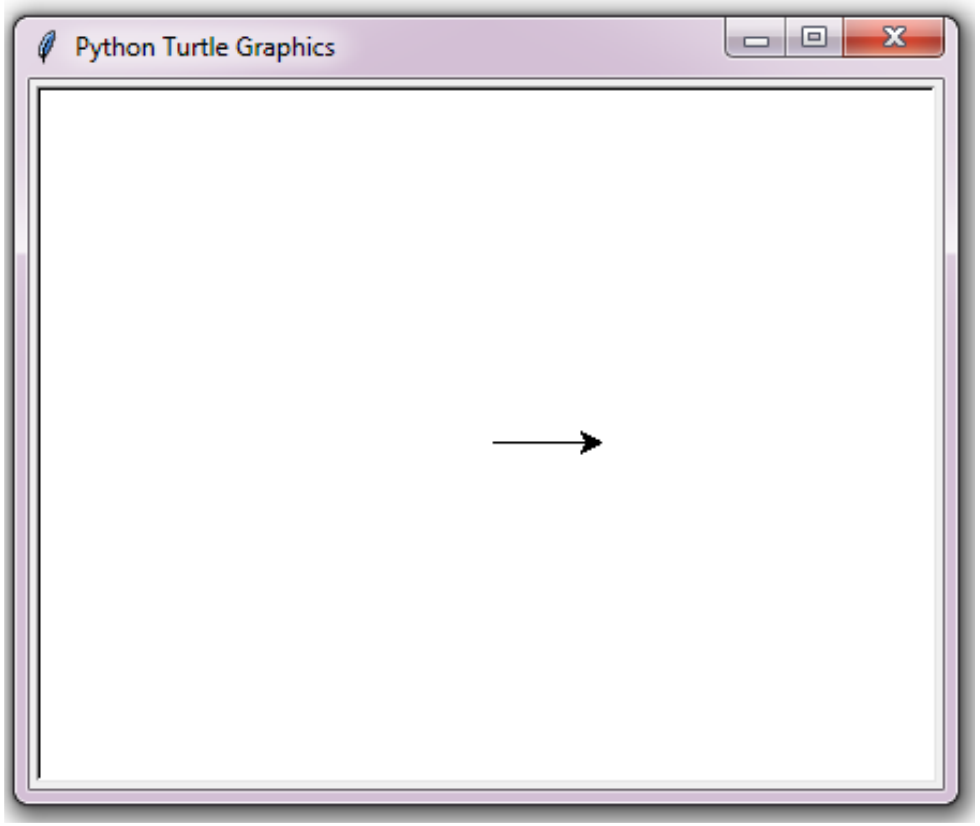
Začneme želví grafikou ...

- https://www.umimeinformatiku.cz/python-zelva
- Jak na to:
  - 1) Běžte na stránku https://www.umimeto.org/
  - 2) Vytvořte si účet. Zaregistrujte se pod svým reálným jménem, e-mail uveďte ve tvaru UČO@mail.muni.cz.
    - I pokud jste systém Umíme informatiku již dříve používali, je nutné si vytvořit nový učet přesně v tomto tvaru, bodování úloh proběhne strojově za využití UČO.)
  - 3) Po vytvoření účtu klikněte na 'Jsem žák', resp. později na 'Připojení žáka do třídy' v menu k účtu) a připojte se skrze heslo:
    - IB113-podzim-2026

Želví grafika - pohyb

- forward(x) – pohyb o x pixelů v daném směru

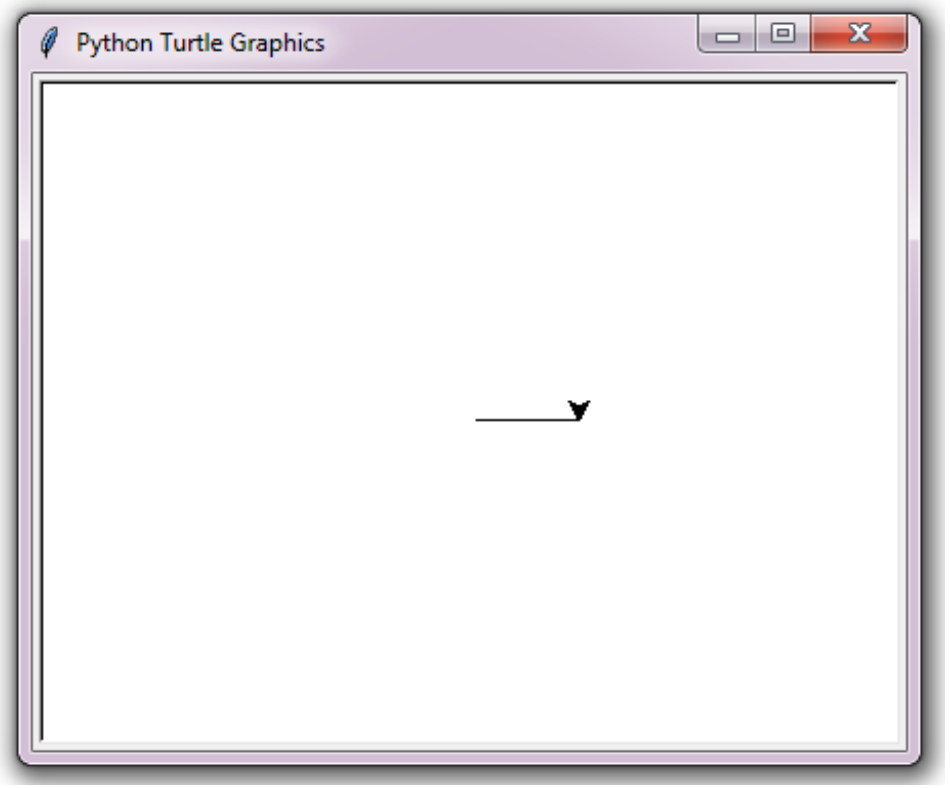
forward(50)



Želví grafika - otočení

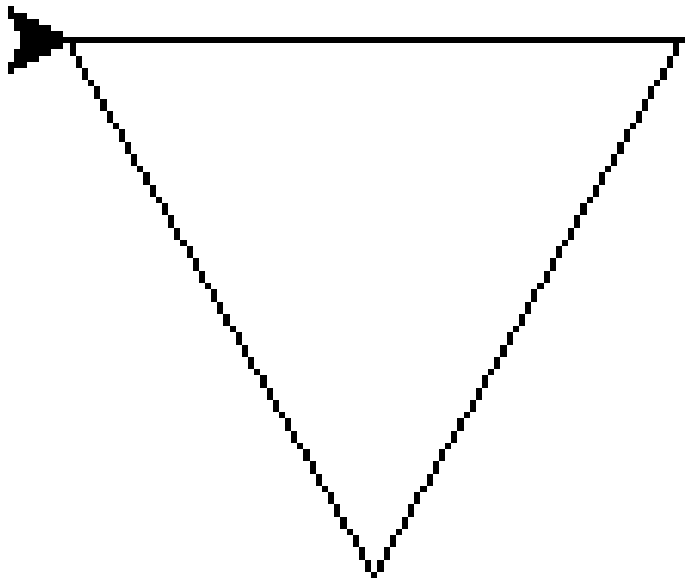
- right(x) - rotace o x stupňů doprava
- left(x) - rotace o x stupňů doleva

right(90)



Želví grafika – trojúhelník poprvé

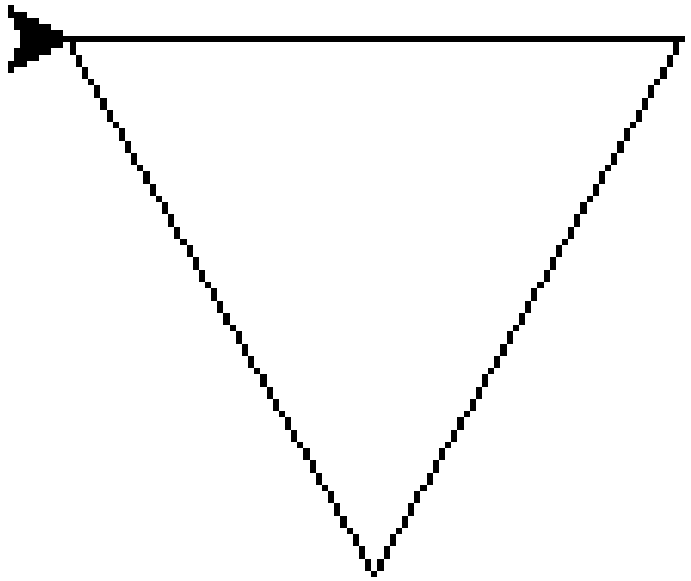
```
forward(100)
right(120)
forward(100)
right(120)
forward(100)
right(120)
```



Želví grafika – trojúhelník s cyklem

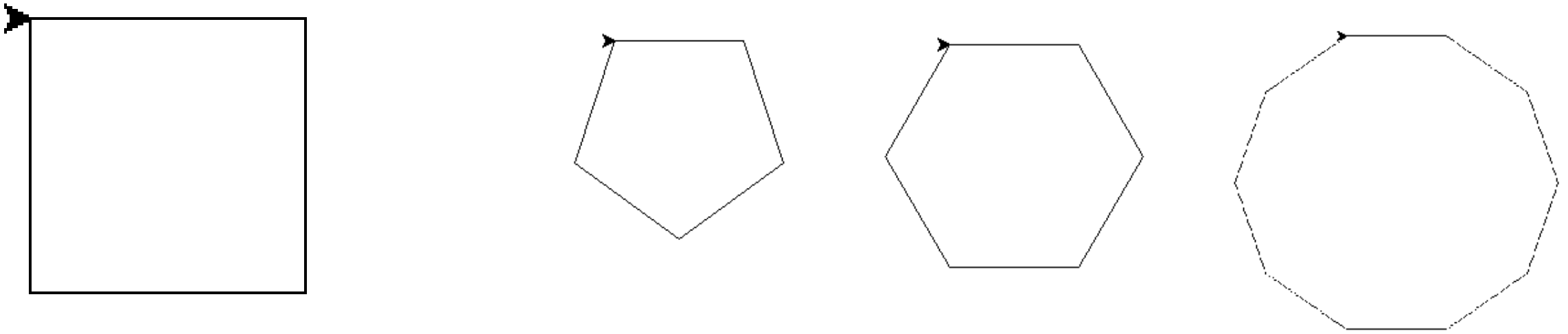
- Příkaz for i in range(3): způsobí 3 opakování

```
for i in range(3):
        forward(100)
        right(120)
```



A co teď?

- https://www.umimeinformatiku.cz/python-zelva
- Udělejte co nejvíce úkolů za 30 minut



Seznámení s IDE Thonny

- Spustit Thonny
- File -> New
- Napište program "Hello world!"
  - print ("Hello world!")
- Spusťte program
  - Bude nutné uložit soubor s kódem
- Pozn. Dalo by se použít i přímo shell (dole), ale doporučuji použít samostatný soubor (budeme to takto používat celý semestr)

Základní konstrukce

- Vypsat číslo 1
  - print ("1")
- Vypsat čísla od 1 do 3
  - print ("1")
  - print ("2")
  - print ("3")
- Ale jak vypsat čísla od 1 do 1000?

Základní konstrukce – proměnné (1)

- Udržují hodnotu
- Udržovaná hodnota se může měnit – proto proměnné
- Typy:
  - Číselné: int, float, ...
  - Pravdivostní hodnota (bool)
  - Řetězec (string)
  - Seznam
  - Slovník
  - ...

Základní konstrukce – proměnné (2)

- Přiřazení =
  - x = 3 znamená „přiřaď do x hodnotu 3“
- Test na rovnost ==
  - x == 3 znamená „otestuj zda v x je hodnota 3“
- Častou chybou je záměna = a ==
- Operace
  - +, -, *, /, and, or
  - Umocňování pomocí **
  - Dělení se zbytkem % (modulo)
  - Celočíselné dělení: //
  - Zkrácený zápis „y += 5“ znamená „y = y + 5“

Základní konstrukce – podmíněný příkaz

```
if <podmínka>:
        příkaz1
else:
        příkaz2
```

- Podle toho, zda platí podmínka, se provede jedna z větví
- Podmínka – typicky výraz nad proměnnými
- else větev je nepovinná
- Vícenásobné větvení: if - elif - ... - else

Úkol

- Vypište, zda je v proměnné x uloženo sudé číslo, liché číslo nebo 0.
  - Přiřaďte x nějakou hodnotu
  - Vytvořte podmíněný příkaz
  - Jakou početní operaci použijete?

Základní konstrukce – cyklus (1)

- Opakované provádění sekvence příkazů
- Známý počet opakování cyklu
- Příkaz for

```
for i in range(n):
    příkazy
```

- range(n) – interval od 0 do n-1 (tj. n opakování)

Základní konstrukce – cyklus (2)

- Neznámý počet opakování cyklu
- Příkaz while
- Opakuj dokud platí podmínka

```
while <podmínka>:
        příkazy

f = 10
while f > 0:
        f -= 2
        print(f)
```

Úkol

- Vypsat čísla od 1 do 50 pomocí cyklu while a for.
  - HINT: pokud chcete, aby byla čísla vypsána na jeden řádek, použijte syntaxi print(to_co_chci_tisknout, end=" ")
- Upravit cykly tak, aby vypsaly jen čísla dělitelná pěti?